Modeling Malware Concept Drift via Continuous-Time Neural Differential Equations

1. The Real-World Problem: Concept Drift in the Malware Landscape

The Adversarial Arms Race and Classifier Degradation

The deployment of machine learning within cybersecurity has historically been characterized by an ongoing and highly asymmetric adversarial arms race. Security vendors deploy high-capacity models—ranging from gradient-boosted decision trees to deep neural networks—trained on massive corpora of benign and malicious Portable Executable (PE) files. In controlled, static evaluations, these models routinely achieve classification accuracy and F1 scores exceeding 0.99[1]. However, post-deployment performance inevitably undergoes a catastrophic degradation, sometimes dropping by as much as 50% in realistic longitudinal evaluations[3].

This degradation is not a symptom of algorithmic instability, inadequate model capacity, or poor hyperparameter tuning; rather, it is a direct consequence of the adversary's continuous evolution. Attackers deliberately mutate source code, adopt novel packing algorithms, alter structural metadata, implement reflective loading, and utilize polymorphic engines to evade detection[5]. Consequently, the statistical relationship between the extracted features of a binary and its true label changes over time. This phenomenon is formally defined in machine learning literature as concept drift, presenting one of the most significant barriers to sustainable, automated threat detection[5].

Formalizing Concept Drift in Cybersecurity

In standard supervised learning, the fundamental mathematical assumption is that the joint probability distribution of the input features X and labels Y, denoted as $P(X, Y)$, remains stationary across both the training and deployment phases. Concept drift represents the severe violation of this core assumption. Within the context of malware classification, this non-stationarity manifests in multiple distinct modalities, each presenting unique challenges to statistical modeling[5].

The various forms of drift require precise definitions to understand why static models fail and why continuous-time modeling is mathematically necessary.

Drift Classification	Mathematical Definition	Cybersecurity Context
Real Concept Drift	$P_{t_1}(Y\|X) \neq P_{t_2}(Y\|X)$	The decision boundary shifts. A code pattern or API

		call sequence historically associated with benign administrative software is co-opted by a novel ransomware family, altering the conditional probability of maliciousness.
Virtual Concept Drift	$P_{t_1}(X) \neq P_{t_2}(X)$	The underlying feature distribution changes without altering the decision boundary. Attackers shift their infrastructure, utilizing different compilers or packing engines that alter the structural entropy of the PE file.
Gradual Drift	Slow, continuous variation in $P(X, Y)$	Polymorphic engines slowly randomize binary structures, causing incremental, continuous deviations in byte-level histograms and string statistics over a period of months.
Sudden (Step) Drift	Abrupt, discontinuous shift in $P(X, Y)$	A zero-day vulnerability is published, or a massive geopolitical event triggers an immediate, massive shift in malware lures, payloads, and delivery mechanisms.

Evaluation Biases and the Illusion of Solved Problems

The impact of concept drift has been systematically understated in historical cybersecurity literature due to pervasive experimental biases. Research utilizing the TESSERACT evaluation framework highlights that standard k-fold cross-validation inherently leaks future knowledge into the training phase[4]. Because malware families evolve chronologically, randomly sampling a

dataset to create training and testing splits virtually guarantees that future variants of a malware family will be included in the training data. This temporal bias artificially inflates model performance, giving a false impression of robustness[3].

Furthermore, spatial bias occurs when the base rate of malware to benign software in the testing dataset does not reflect real-world deployment conditions[1]. When the strict spatio-temporal constraints defined by TESSERACT are rigorously enforced, the true vulnerability of static models to concept drift becomes glaringly apparent[12]. This exposes the massive financial and operational costs associated with continually retraining production models, which currently require analysts to manually label drifting samples to update static boundaries[9].

To quantify this degradation objectively, the Area Under Time (AUT) metric provides a definitive measure of robustness. Based on the trapezoidal rule, AUT captures a classifier's robustness to time decay by integrating the Area Under the Receiver Operating Characteristic (AUROC) over a sequence of future temporal testing blocks[4]. The failure of current models to maintain a stable AUT score necessitates a paradigm shift in how temporal adversarial dynamics are modeled.

2. Limitations of Prevailing Detection Architectures

The Static Classifier Trap

Traditional machine learning paradigms, including Random Forests, XGBoost, LightGBM, and standard Multilayer Perceptrons (MLPs), learn a frozen decision boundary during the training phase. These models operate under the implicit assumption that the topological relationship between malicious and benign clusters in the high-dimensional feature space will remain constant indefinitely.

When applied to a temporally shifting landscape, static models fail because they lack an internal mechanism to account for the trajectory of adversarial evasion. If a model is trained on data from a specific eight-month temporal window, the decision boundary is optimized exclusively for the feature distributions present within that timeframe. By the twelfth month, virtual and real drift will have caused the malware clusters to migrate to entirely new regions of the feature space[4]. The static classifier, completely oblivious to this migration, continues to apply the obsolete decision boundary. This results in a surge of false negatives as novel malware evades detection, and a corresponding spike in false positives as benign software adopts structural features previously correlated with malware.

The Inadequacies of Discrete Sequence Models

To incorporate temporal awareness, sequence models such as Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs), and Gated Recurrent Units (GRUs) have been frequently proposed. While highly effective in domains with regularly sampled time series—such as audio processing, natural language, or financial tick data—these discrete-time models exhibit critical vulnerabilities when applied to malware concept drift[16].

The primary inadequacy stems from the assumption of fixed time steps. Standard RNNs assume that observations arrive at uniform, discrete intervals. However, malware evolution and sample collection are highly irregular. A specific threat actor might release dozens of variants of

a trojan within a single week during an active campaign, followed by months of silence before launching a newly obfuscated variant[16]. Discrete sequence models cannot naturally encode the variable elapsed time between these observations without resorting to artificial binning or time-delta concatenation, both of which severely degrade representational fidelity[18].
Furthermore, sequence models rely on discrete latent state transitions. In an RNN, the hidden state h_t is updated via a discrete mapping: $h_{t+1} = f(h_t, x_{t+1})$. Between observation time t and $t+1$, the model assumes the system state remains entirely static[17]. Yet, concept drift is an active, continuous process occurring outside the observation window. The adversary's codebase is evolving and the feature distribution is flowing smoothly through the latent space even when defenders are not actively observing new samples. Discrete transitions fail to capture this continuous underlying dynamic.

3. Core Insight: Continuous-Time Dynamics and Neural ODEs

Shifting from Discrete Boundaries to Continuous Vector Fields

The profound limitations of static boundaries and discrete transitions necessitate a fundamental architectural shift: modeling the continuous-time dynamics of malware feature evolution. This is achieved through the application of Neural Ordinary Differential Equations (Neural ODEs). First formalized by Chen et al. (2018), Neural ODEs replace the discrete sequence of hidden layers in a deep neural network with a continuous formulation, parameterizing the derivative of the hidden state using a neural network[20].
Instead of learning a static boundary to separate clusters at a frozen point in time, the Neural ODE learns a continuous vector field that dictates the rules governing how a malware sample's features change over time. The continuous dynamics are defined by the initial value problem:

$$\frac{dh(t)}{dt} = f_\theta(h(t), t)$$

Where $h(t)$ represents the latent representation of the malware sample at time t , and the function f_θ , parameterized by a neural network, predicts the instantaneous rate of change (the velocity) of the latent state[18]. To predict the future state of a malware family, the model integrates this vector field forward in time using an off-the-shelf numerical ODE solver:

$$h(t_1) = h(t_0) + \int_{t_0}^{t_1} f_\theta(h(\tau), \tau) d\tau$$

By learning the underlying currents of the feature space—such as the trajectory of increasing entropy due to novel packing engines, or the gradual shift in imported DLLs due to new evasion tactics—the Neural ODE can project a historical malware sample into the future. This enables

the model to effectively anticipate where the malware cluster will move, providing proactive classification capabilities that maintain robustness against gradual concept drift[7].

Ensuring Theoretical Validity: The Necessity of Neural Jump ODEs (NJODEs)

While standard Neural ODEs excel at modeling smooth, continuous trajectories, relying exclusively on a continuous framework is theoretically insufficient for the realities of cybersecurity. Concept drift in malware is characterized by a combination of continuous gradual drift (e.g., polymorphic iteration) and sudden step-drift (e.g., the rapid exploitation of a newly disclosed zero-day vulnerability, or a sudden change in global geopolitical events dictating new attacker objectives)[24].

Because ODE trajectories cannot cross each other and are mathematically bound by the Picard-Lindelöf theorem, standard Neural ODEs struggle to model sudden, discontinuous jumps in the state space[18]. An ODE assumes the evolution of features is a smooth homeomorphism, which breaks down when a completely novel malware family appears instantaneously in the wild. Therefore, to ensure the proposed solution is valid, the architecture must be augmented from a standard Latent ODE to a Neural Jump Ordinary Differential Equation (NJODE) framework[25].

NJODEs are continuous-time hybrid models that alternate between continuous flows (governed by the Neural ODE) and instantaneous state updates (jumps) triggered by specific stochastic events[25]. This allows the model to capture the smooth evolution of an existing malware family while simultaneously accommodating the abrupt emergence of a completely novel feature distribution.

In the NJODE framework, continuous evolution is maintained between observations. However, at the moment a new, highly anomalous malware sample X_{t_i} is observed, the latent state undergoes a memoryless jump update:

$$h_{t_i} = \rho_\phi(X_{t_i})$$

Where ρ_ϕ is a distinct neural network that maps the sudden observation directly into a new position in the latent space, effectively resetting the initial conditions for the subsequent continuous ODE flow[25]. This critical upgrade ensures the model remains mathematically sound and highly performant against the erratic, discontinuous nature of real-world adversarial behavior.

4. Mathematical Formulation of the Drift-Resilient Architecture

The Latent NJODE Pipeline

To process high-dimensional malware features and integrate them through continuous time, the architecture requires an intricate encoder-solver-decoder pipeline, heavily inspired by Latent ODE formulations for irregularly sampled time series[19].

1. Dimensionality Reduction (Encoder):

Given a high-dimensional feature vector $x \in \mathbb{R}^{2381}$ extracted from a PE file, a neural network encoder maps this into a compact, lower-dimensional latent space $z \in \mathbb{R}^{d}$ (e.g., $d = 64$). This dimensionality reduction filters out irrelevant structural noise and creates a dense representation where continuous mathematical flows are tractable.

$$z_{t_0} = \text{Encoder}_{\omega}(x_{t_0})$$

2. Continuous Integration (ODE Solver):

The initial latent state z_{t_0} is treated as the initial condition for the differential equation. To predict the state of the malware family at a future time t_f, the state is evolved continuously across the temporal gap:

$$z(t_f) = z_{t_0} + \text{ODESolve}(f_\theta, z_{t_0}, (t_0, t_f))$$

The solver utilizes adaptive step-size algorithms, such as the Dormand-Prince (dopri5) method, which dynamically adjust the evaluation step size to explicitly trade off between numerical precision and computational speed[20].

3. Jump Mechanics for Sudden Drift: If a significant distributional shift is detected—such as the arrival of an out-of-distribution sample that defies the current flow field—the NJODE bypasses the continuous integral and applies the instantaneous jump map ρ to reset the latent trajectory, allowing the system to rapidly adapt to a novel threat landscape without being bound by previous continuous dynamics[28].

4. Classification (Decoder):

The temporally evolved latent state $z(t_f)$ is subsequently passed through a standard Multilayer Perceptron decoder to yield the final classification logit indicating whether the evolved state represents a benign or malicious entity.

$$\hat{y} = \text{Softmax}(\text{Decoder}_{\psi}(z(t_f)))$$

The Adjoint Sensitivity Method and Memory Efficiency

A primary historical barrier to implementing continuous-time deep learning was the immense memory cost associated with backpropagation. Differentiating directly through the operations of a numerical ODE solver requires storing every intermediate state evaluated during the forward integration. This results in a memory complexity of $\mathcal{O}(L)$, where L is the number of steps taken by the solver. During complex optimization, this rapidly leads to out-of-memory errors on standard hardware[18].

The mathematical solution that renders this continuous architecture highly scalable is the Adjoint Sensitivity Method[18]. Instead of storing intermediate states, the adjoint method treats the ODE solver as a black box and solves a second, augmented ODE backward in time to compute the gradients of the loss function with respect to the initial state and the model parameters.

Defining the adjoint state as $a(t) = \partial \mathcal{L} / \partial z(t)$, its continuous dynamics are dictated by the equation:

$$\frac{da(t)}{dt} = -a(t)^\top \frac{\partial f_\theta(z(t), t)}{\partial z}$$

By utilizing the adjoint method, the memory cost of the backward pass drops from $\mathcal{O}(L)$ to $\mathcal{O}(1)$ —remaining completely constant and independent of the number of steps taken by the solver[18]. This mathematical elegance ensures that training the Latent NJODE model is highly memory-efficient, permitting implementation on consumer-grade GPUs without the need for distributed computing clusters.

Handling Imbalanced Data: Focal Loss Formulation

Malware datasets are inherently imbalanced, and concept drift tends to disproportionately affect minority classes (e.g., highly targeted, rare advanced persistent threat payloads). Standard Cross-Entropy loss heavily weights the easily classified majority samples, allowing the model to achieve low loss by simply memorizing static benign software patterns.
To force the continuous dynamics to respect the intricate trajectories of rare, hard-to-classify polymorphic variants, the model utilizes a Focal Loss formulation:

$$\mathrm{FL}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t)$$

Setting the focusing parameter $\gamma = 2.0$ drastically down-weights the loss contributed by simple, static files, forcing the ODE vector field to dedicate its representational capacity toward learning the complex temporal flow of drifting malware clusters located near the highly contested decision boundary.

5. Dataset Foundations: Evaluating Continuous Dynamics

To effectively validate continuous-time domain adaptation and prove the efficacy of Neural Jump ODEs against malware concept drift, the underlying dataset must possess explicit timestamp metadata, realistic chronological ordering, and a sufficient temporal span. The BODMAS (Blue Hexagon Open Dataset for Malware Analysis) repository is ideally suited for this stringent requirement, surpassing alternatives like EMBER or SOREL-20M in temporal granularity[24].

Statistical Profile and Structural Feature Space

The BODMAS dataset contains 134,435 samples—comprising 57,293 malware and 77,142 benign PE files—collected over a contiguous 14-month window from August 2019 to September 2020[33]. The explicit timestamp associated with each sample permits the precise modeling of continuous-time evolution necessary for ODE integration.

Each sample is represented as a pre-extracted 2381-dimensional numeric feature vector. These vectors are generated using the LIEF (Library to Instrument Executable Formats) framework, which parses the structural integrity of the Windows Portable Executable format. The 2381 features encompass multiple orthogonal views of the binary, rendering the dataset highly robust for analyzing complex feature drift[31]:

- **PE Header Metadata:** Metrics regarding the number of sections, entry point coordinates, and file alignment configurations.
- **Sectional Entropy:** The statistical entropy of distinct PE sections (e.g., .text, .data). This is vital for drift modeling, as the continuous adoption of novel cryptographic packers by adversaries heavily influences entropy distributions.
- **Import Address Table (IAT):** Statistical summaries of the imported DLLs and specific API calls (e.g., kernel32.dll, ws2_32.dll), which reflect ongoing changes in the functional capabilities and payloads of the evolving malware.
- **Byte-Level Histograms:** Raw statistical distributions of the compiled byte code, offering a low-level view of polymorphic mutations that occur over time[31].

The Natural Experiment of COVID-19 Concept Drift

The 14-month span of the BODMAS dataset encapsulates one of the most significant periods of cyber-adversarial adaptation in modern history. The first half of the dataset (August 2019 to February 2020) establishes a pre-pandemic baseline of malware tactics, dominated by standard variants of Emotet and various banking trojans[33].

However, beginning in March 2020, the onset of the global COVID-19 pandemic catalyzed a massive, documented shift in malware tactics, generating a surge of novel phishing lures, healthcare-targeted ransomware, and entirely new delivery infrastructures. This provides an unprecedented natural experiment for evaluating concept drift. The suddenness of this transition is exactly why the proposed upgrade to Neural Jump ODEs is theoretically necessary; the continuous vector field models the gradual drift of existing families, while the discrete jump mechanism immediately accommodates the discontinuous emergence of pandemic-themed payloads without fracturing the ODE solver[24].

Comparative Baselines: EMBER and SOREL-20M

While datasets like EMBER (2018) and SOREL-20M provide massive scale—with SOREL containing 20 million samples—they lack the rigorous, dense temporal granularity required for high-fidelity ODE integration[35]. TESSERACT evaluations on Android datasets have proven that scale alone does not mitigate the effects of concept drift if the temporal evaluation windows are not strictly enforced[12]. BODMAS, explicitly designed to highlight temporal family evolution, serves as the superior empirical foundation for proving the efficacy of continuous-time dynamics.

6. Experimental Protocol and Rigorous Evaluation Framework

An optimal theoretical model is only as verifiable as its experimental methodology. To prevent the spatial and temporal biases outlined by the TESSERACT framework[3], the experimental protocol enforces strict chronological partitioning, ensuring that the model is never exposed to future feature distributions during the training phase.

Sequential Train-Validation-Test Splitting

The dataset is partitioned chronologically based on the precise monthly timestamps inherent to BODMAS to simulate realistic deployment conditions[37]:

- **Training Set (Months 0–7):** August 2019 through March 2020. This allows the model to learn the baseline vector field and the initial shocks of the early pandemic drift.
- **Validation Set (Months 8–9):** April 2020 through May 2020. This near-future data is strictly used for hyperparameter tuning, early stopping, and calibrating the ODE solver tolerances (e.g., setting the rtol and atol thresholds). Under no circumstances is the model's performance on this set conflated with final testing.
- **Testing Set (Months 10–13):** June 2020 through September 2020. This far-future data represents a severe concept drift scenario. The evaluation tests the model's ability to extrapolate its learned continuous dynamics across an unobserved temporal gap.

The Five-Stage Experimental Hierarchy

To systematically prove the superiority of continuous-time dynamics in combating malware drift, the evaluation executes a five-stage experimental protocol:

Experiment 1: Static Baseline Profiling State-of-the-art static classifiers—specifically XGBoost, LightGBM, and an MLP matching the exact parameter count of the NJODE—are trained strictly on Months 0–7. They are evaluated on the testing set to establish the performance ceiling of non-temporal modeling. The observed decay in Area Under the Receiver Operating Characteristic (AUROC) between the validation and test sets quantifies the exact magnitude of the concept drift present in the data[2].

Experiment 2: NJODE Temporal Integration

The Neural Jump ODE model is trained on the exact same Months 0–7 data. Because the architecture actively learns the drift trajectory as a physical dynamic system, its evaluation on Months 10–13 should exhibit a significantly attenuated performance drop compared to the static baselines.

Experiment 3: Longitudinal Drift Quantification (AUT) To adhere to the highest standards of cybersecurity evaluation, the results are tracked dynamically over time using the Area Under Time (AUT) metric[4]. Rather than reporting a single aggregate score for the entire testing block, performance is evaluated sequentially on each individual future month. By plotting AUROC against continuous time (Months 8 through 13), a visual and statistical narrative is established. While static baselines will display a steep negative slope indicating performance decay, a successful NJODE will maintain a near-flat trajectory, definitively proving drift resilience over an extended operational horizon[4].

Experiment 4: Architectural Ablation Studies To isolate the specific mechanisms responsible for resilience, targeted ablations are performed. This includes decoupling the ODE solver—routing the encoder directly to the decoder—to prove that the continuous integration, and not merely the feature extraction network, is responsible for the performance retention. Additionally, solver ablations comparing adaptive solvers (dopri5) against fixed-step solvers (rk4) clarify the crucial trade-off between integration precision and computational overhead, quantifying how solver choice impacts the detection of rapidly evolving malware families[18].
Experiment 5: Latent Topological Visualization Because Neural ODEs are inherently generative mappings, their latent space can be directly visualized to ensure the learned dynamics correspond to physical reality. Using dimensionality reduction algorithms like UMAP, the initial embedded states (z_0) are plotted against the chronologically evolved states (z_{t_f}). If the continuous flow field is mathematically sound, the evolved states should exhibit distinct chronological separation, visually confirming that the network has mathematically untangled the complex topological knot of temporal concept drift[38].

7. Identifying and Mitigating Systemic Risks in ODE Training

While the theoretical foundation of Neural ODEs is mathematically elegant, their practical deployment on highly non-stationary datasets introduces distinct engineering risks, particularly regarding numerical stability and computational overhead. Overcoming these limitations is necessary for a robust, deployable cybersecurity solution.

The NFE Explosion and Stiff ODEs

The most prominent systemic risk in continuous-time deep learning is the explosion of the Number of Function Evaluations (NFE). As the model trains, the continuous flow it attempts to map often becomes highly complex, contorted, and nonlinear as it tries to separate increasingly sophisticated malware samples from benign software. When an ordinary differential equation exhibits widely varying timescales—where certain latent components change rapidly while others remain stable—the system is mathematically classified as a Stiff ODE[39].
Standard adaptive explicit solvers, such as the widely utilized dopri5, struggle severely with stiffness. To maintain numerical stability and satisfy predefined error tolerances, the explicit solver is forced to take infinitesimally small step sizes. This causes the NFE to skyrocket exponentially[26]. The resulting NFE explosion leads to drastically increased training times, creates massive latency during inference, and can ultimately cause backpropagation via the adjoint method to fail due to accumulated numerical rounding errors[43].
To mitigate this critical bottleneck, the research protocol must dynamically monitor stiffness. If stiffness is encountered during training on the BODMAS dataset, the architecture must transition from explicit solvers to single-step implicit schemes. Implicit methods provide a vastly superior framework for handling stiff dynamics without fracturing the step size, allowing the neural ODE to accurately learn stiff systems without succumbing to stability collapse[39]. Furthermore, implementing kinetic or Jacobian regularization techniques can heavily penalize highly contorted vector fields, forcing the model to learn smoother, more easily integrated

dynamics that keep NFE manageable without sacrificing predictive power[27].

Alternative Generative Approaches: Continuous Normalizing Flows (CNFs)

If the discriminative Latent NJODE model struggles to capture the sheer diversity of virtual drift across the 2381-dimensional structural feature space, the methodology can pivot toward modeling the exact probability density function of the malware features using Continuous Normalizing Flows (CNFs)[45].

CNFs construct an exact, invertible mapping between a simple base distribution (e.g., a standard Gaussian) and the highly complex, non-stationary data distribution of the drifting malware[21]. By leveraging the instantaneous change of variables formula—which elegantly requires only the trace of the Jacobian rather than the computationally prohibitive determinant—CNFs can track the exact likelihood of a malware sample over continuous time[18]. Recent advances in flow matching bypass the need for log-density evaluation at every solver step, training the network directly via regression to analytically prescribed velocity fields. This dramatically accelerates training and provides non-asymptotic convergence bounds[46]. While computationally heavier than a standard classifier, CNFs utilizing modern flow-matching techniques offer an incredibly rigorous fallback strategy for modeling severe concept drift with exact likelihood quantification[45].

8. Foundational Literature and Future Research Directions

The architecture defined in this report represents a novel synthesis that exists at the precise intersection of three distinct, cutting-edge domains: continuous-time deep learning, rigorous spatio-temporal cybersecurity evaluation, and longitudinal malware dataset benchmarking. The mathematical foundation rests heavily upon the introduction of Neural ODEs by Chen et al. (2018), which revolutionized the application of continuous-depth networks and the adjoint sensitivity method[20]. This continuous paradigm was subsequently adapted for irregularly sampled time series via Latent ODEs by Rubanova et al. (2019)[19]. However, the specific vulnerability of pure ODEs to sudden, discontinuous state changes necessitated the integration of Neural Jump ODEs (NJODEs), introduced by Herrera et al. (2020), providing the critical mathematical machinery to handle sudden malware step-drift efficiently[17].

Simultaneously, the experimental rigor demanded by this architecture is entirely derived from the TESSERACT framework formulated by Pendlebury et al. (2019)[2]. Prior to TESSERACT, the cybersecurity community fundamentally misunderstood the severity of concept drift due to pervasive spatial and temporal experimental biases. The introduction of time-aware splits and the Area Under Time (AUT) metric forces models to prove genuine predictive capability against future data[4]. Finally, the execution of this experiment relies entirely on the BODMAS dataset compiled by Yang et al. (2021), which provides the densely timestamped, realistic malware vectors necessary for ODE integration[24].

Prior literature has predominantly addressed malware concept drift via active learning and discrete retraining loops—periodically labeling new, anomalous samples and updating the static

decision boundary manually[5]. Similarly, while continuous-time networks like Latent ODEs and NJODEs have seen vast success in healthcare time-series analysis and optimal financial filtering[25], their application to the high-dimensional structural features of drifting malware represents a largely unexplored frontier.

Future research directions must focus on expanding the continuous-time framework to graph-structured data. Malware execution traces can be naturally represented as Control Flow Graphs (CFGs) or API call graphs. Integrating continuous dynamics with Graph Neural Networks (GNNs) could allow the ODE to model not just the drift of structural features, but the actual evolution of malicious behavioral execution paths over time[10].

9. Conclusions and Strategic Implications

The pervasive phenomenon of concept drift dictates that static machine learning models deployed in cybersecurity environments are inherently depreciating assets. As adversaries continuously mutate code, leverage new packing engines, and adapt to global events, the static decision boundaries protecting network infrastructure rapidly become obsolete, necessitating costly manual retraining and leaving critical operational windows vulnerable to novel threats.

By upgrading the static detection paradigm to a Neural Jump Ordinary Differential Equation architecture, this framework ceases to view malware as static points scattered in a vacuum. Instead, it correctly interprets adversarial evolution as a fluid, dynamic physical system—a continuous vector field punctuated by rapid, stochastic jumps triggered by novel exploit kits or geopolitical shifts.

The integration of the BODMAS dataset ensures that the model is trained on realistic, densely timestamped adversarial behavior, while the strict adherence to the TESSERACT evaluation framework guarantees that the model's resilience to drift is mathematically validated against strict temporal gaps. Furthermore, by identifying and preparing for the risks associated with stiff ODEs and the NFE explosion, the architecture maintains computational feasibility for real-world deployment.

If validated according to the rigorous metrics proposed herein, the capability to mathematically forecast the trajectory of adversarial evolution will fundamentally alter the defensive posture of the cybersecurity industry. It transitions automated threat detection from an endless, reactive patching cycle into a proactive, predictive science, where the continuous evolution of the adversary is actively integrated into the fabric of the defensive model.

Works cited

1. TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time | USENIX, https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury
2. [1807.07838] TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time - arXiv, https://arxiv.org/abs/1807.07838
3. TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time (Extended Version) - arXiv, https://arxiv.org/html/2402.01359v2
4. TESSERACT: Eliminating Experimental Bias in Malware Classification across Space

and Time - USENIX, https://www.usenix.org/system/files/sec19-pendlebury.pdf
5. MORPH: Towards Automated Concept Drift Adaptation for Malware Detection - arXiv, https://arxiv.org/html/2401.12790v1
6. Empirical Evaluation of Concept Drift in ML-Based Android Malware Detection, https://www.researchgate.net/publication/394121805_Empirical_Evaluation_of_Concept_Drift_in_ML-Based_Android_Malware_Detection
7. Detecting Concept Drift in Evolving Malware Families Using Rule-Based Classifier Representations - arXiv, https://arxiv.org/html/2604.22629v1
8. [2502.14135] Cluster Analysis and Concept Drift Detection in Malware - arXiv, https://arxiv.org/abs/2502.14135
9. Going Proactive and Explanatory Against Malware Concept Drift - arXiv, https://arxiv.org/html/2405.04095v1
10. Behavioral Drift–Aware Malware Detection: A Survey of Graph Neural Networks for Explainable and Operational Deployment - isrdo, https://isrdo.org/journal/SRJSET/currentissue/htmlview/behavioral-driftaware-malware-detection-a-survey-of-graph-neural-networks-for-explainable-and-operational-deployment
11. TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time - King's College London Research Portal, https://kclpure.kcl.ac.uk/portal/en/publications/tesseract-eliminating-experimental-bias-in-malware-classification/
12. (PDF) Breaking Out from the TESSERACT: Reassessing ML-based Malware Detection under Spatio-Temporal Drift - ResearchGate, https://www.researchgate.net/publication/393184496_Breaking_Out_from_the_TESSERACT_Reassessing_ML-based_Malware_Detection_under_Spatio-Temporal_Drift
13. Breaking Out from the Tesseract: Reassessing ML-based Malware Detection under Spatio-Temporal Drift - BIFOLD Berlin, https://www.bifold.berlin/impact-transfer/publications/view/publication-detail/breaking-out-from-the-tesseract-reassessing-ml-based-malware-detection-under-spatio-temporal-drift
14. Revisiting Concept Drift in Windows Malware Detection: Adaptation to Real Drifted Malware with Minimal Samples - NDSS Symposium, https://www.ndss-symposium.org/wp-content/uploads/2025-830-paper.pdf
15. TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time (Extended Version) - arXiv, https://arxiv.org/html/2402.01359v1
16. [2212.13714] Continuous Depth Recurrent Neural Differential Equations - arXiv, https://arxiv.org/abs/2212.13714
17. NEURAL JUMP ORDINARY DIFFERENTIAL EQUATIONS: CONSISTENT CONTINUOUS-TIME PREDICTION AND FILTERING - arXiv, https://arxiv.org/pdf/2006.04727
18. Neural ODEs: Continuous-Depth Deep Learning Models, https://hunterheidenreich.com/notes/machine-learning/generative-models/neural-odes/
19. Latent Ordinary Differential Equations for Irregularly-Sampled Time Series,

https://proceedings.neurips.cc/paper/2019/hash/42a6845a557bef704ad8ac9cb4461d43-Abstract.html
20. [1806.07366] Neural Ordinary Differential Equations - arXiv, https://arxiv.org/abs/1806.07366
21. Neural Ordinary Differential Equations, https://papers.neurips.cc/paper/7892-neural-ordinary-differential-equations.pdf
22. [2103.12413] Neural ODE Processes - arXiv, https://arxiv.org/abs/2103.12413
23. [2006.04727] Neural Jump Ordinary Differential Equations: Consistent Continuous-Time Prediction and Filtering - arXiv, https://arxiv.org/abs/2006.04727
24. BODMAS: An Open Dataset for Learning based Temporal Analysis of PE Malware - Gang Wang, https://gangw.cs.illinois.edu/DLS21_BODMAS.pdf
25. Neural Jump ODEs: Continuous-Time Hybrid Models - Emergent Mind, https://www.emergentmind.com/topics/neural-jump-ordinary-differential-equations-njodes
26. Augmented Neural ODEs, http://papers.neurips.cc/paper/8577-augmented-neural-odes.pdf
27. Chapter 3: Neural Ordinary Differential Equations - Deep Implicit Layers, http://implicit-layers-tutorial.org/neural_odes/
28. Neural Jump Ordinary Differential Equations: Consistent Continuous-Time Prediction and Filtering - IDEAS/RePEc, https://ideas.repec.org/p/arx/papers/2006.04727.html
29. Neural Jump Stochastic Differential Equations - Cornell: Computer Science, https://www.cs.cornell.edu/~arb/papers/njsde-neurips-2019.pdf
30. Neural Differential Equations for Irregularly Sampled Time Series - Diva Portal, https://uu.diva-portal.org/smash/get/diva2:2050557/FULLTEXT01.pdf
31. BODMAS Malware Dataset - GitHub, https://github.com/whyisyoung/BODMAS
32. BODMAS: An Open Dataset for Learning based Temporal Analysis of PE Malware, https://www.semanticscholar.org/paper/BODMAS%3A-An-Open-Dataset-for-Learning-based-Temporal-Yang-Ciptadi/e374a82544c698a2a8191108a8e6a9a1eaa9f571
33. BODMAS: An Open Dataset for Learning based Temporal Analysis of PE Malware, https://scispace.com/papers/bodmas-an-open-dataset-for-learning-based-temporal-analysis-48tewukpj6
34. BODMAS Malware Dataset - Limin Yang, https://whyisyoung.github.io/BODMAS/
35. [2308.10821] Optimized Deep Learning Models for Malware Detection under Concept Drift, https://arxiv.org/abs/2308.10821
36. The SOREL-20M dataset - Squarespace, https://static1.squarespace.com/static/59b5d23659cc681cdd75f9dc/t/638be02d1c13dd4c9f8d40ca/1670111278636/camlis_2021_harang.pdf
37. POSTER: Enabling Fair ML Evaluations for Security, https://s2lab.cs.ucl.ac.uk/downloads/tesseract-poster-paper.pdf
38. Neural ODE Approach - Emergent Mind, https://www.emergentmind.com/topics/neural-ode-approach
39. Neural ODEs for Stiff Systems: Implicit Single-Step Methods - arXiv, https://arxiv.org/html/2410.05592v1

40. Training stiff neural ordinary differential equations with implicit single-step methods - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC11646139/
41. Stiff neural ordinary differential equations - DSpace@MIT, https://dspace.mit.edu/entities/publication/2a02423c-7791-4b39-9598-a64b0d2a5eda
42. Neural ordinary differential equations for model order reduction of stiff systems - Dipartimento di Matematica - Politecnico di Milano, https://www.mate.polimi.it/biblioteca/add/qmox/53-2024.pdf
43. Research Gaps in Neural ODE. Neural Ordinary Differential Equations... | by Farshad Noravesh | Medium, https://medium.com/@noraveshfarshad/research-gaps-in-neural-ode-8480aaa5661b
44. Taylor-Lagrange Neural Ordinary Differential Equations: Toward Fast Training and Evaluation of Neural ODEs - IJCAI, https://www.ijcai.org/proceedings/2022/0405.pdf
45. Convergence of Continuous Normalizing Flows for Learning Probability Distributions - arXiv, https://arxiv.org/abs/2404.00551
46. Continuous Normalizing Flows (CNF) - Emergent Mind, https://www.emergentmind.com/topics/continuous-normalizing-flows-cnf
47. Normalising Flows and Neural ODEs | Back2Numbers - Emmanuel Rialland, https://emmanuel-r8.github.io/post/2020/09/11/normalising-flows/
48. Neural Jump Ordinary Differential Equations: Consistent Continuous-Time Prediction and Filtering | OpenReview, https://openreview.net/forum?id=JFKR3WqwyXR
49. [2602.08980] When do neural ordinary differential equations generalize on complex networks? - arXiv, https://arxiv.org/abs/2602.08980